

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
Федеральное государственное бюджетное образовательное учреждение высшего
образования
**«СИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ И ИНФОРМАТИКИ»**
(СибГУТИ)
Институт вычислительной техники
Кафедра вычислительных систем

ОТЧЕТ
по практической работе № 5
по дисциплине «Основы разработки на Java»

Выполнил: студент гр. ИС-541
Устюжанин Д. К.

Проверил: доцент кафедры ВС
Нигматулин Р. Р.

Новосибирск 2026

Содержание

Содержание	2
1. Введение	3
1.1 Цель работы	3
1.2 Общая предметная модель	3
2. Работа с базовыми коллекциями Java Collections Framework	4
2.1 Задание 1. Последовательность и порядок в List	4
2.2 Задание 2. Гарантия уникальности и Set	4
2.3 Задание 3. Индексация данных и Map	5
2.4 Задание 4. Организация очередей (Queue)	6
2.5 Задание 5. Безопасность модификации коллекций при обходе	6
3. Обобщенное программирование (Generics)	8
3.1 Задание 6. Обобщенные типы данных (Result<T> и Page<T>).....	8
3.2 Задание 7. Обобщенные методы и утилиты	8
3.3 Задание 8. Ограничения типов и Bounded Wildcards	9
3.4 Задание 9. Практическое применение принципа PECS	9
4. Контракты структур данных и архитектурное проектирование	11
4.1 Задание 10. Соблюдение контракта ключа в хэш-таблицах	11
4.2 Задание 11. Выбор оптимальной структуры данных под нагрузку	11
4.3 Задание 12. Проектирование обобщенного репозитория (Repository)	12
4.4 Задание 13. Построение инвертированного индекса	12
4.5 Задание 14. Реализация LRU-кэша на базе LinkedHashMap	13
4.6 Задание 15. Построение и анализ графа пререквизитов	13
5. Заключение	15
Приложение А. Ключевые листинги программы.....	16

1. Введение

В рамках данной практической работы рассматриваются базовые и продвинутые механизмы работы с каркасом коллекций (Java Collections Framework) и обобщенным программированием (Generics) в языке Java. В отчете представлены решения 15 заданий, охватывающих эффективное использование списков, множеств, отображений, очередей, алгоритмы топологической сортировки, построение LRU-кэша, инвертированного индекса, а также проектирование обобщенных репозитория и соблюдение контрактов equals/hashCode и принципа PECS.

1.1 Цель работы

Освоить работу со стандартными коллекциями Java и механизмами обобщенного программирования:

1. Понять различия в асимптотической сложности операций и внутреннем устройстве ArrayList, LinkedList, HashSet, LinkedHashSet, TreeSet, HashMap, LinkedHashMap, TreeMap, ArrayDeque.
2. Научиться применять правила безопасной модификации коллекций при итерации и избегать ConcurrentModificationException.
3. Овладеть разработкой обобщенных классов и методов (Generics), использованием Bounded Wildcards (? extends T, ? super T) и принципом PECS.
4. Изучить контракты equals/hashCode и их влияние на корректность работы хэш-таблиц.
5. Проектировать специализированные структуры данных (LRU-кэш, инвертированный индекс, граф зависимостей с поиском циклов и топологической сортировкой).

1.2 Общая предметная модель

Все задания выполнены в рамках единой предметной модели учебного центра **CourseHub**:

- **src.CourseId** — value object идентификатора курса, реализующий Comparable<CourseId>.
- **src.CourseCode** — value object кода курса с нормализацией символов (приведение к верхнему регистру, удаление пробелов и разделителей).
- **src.Course** — сущность учебного курса (содержит CourseId, наименование, длительность, популярность и множество тегов Tag). Равенство equals и hashCode определены строго по CourseId.
- **src.Tag** — value object тега с нормализацией имени в нижний регистр.

2. Работа с базовыми коллекциями Java Collections Framework

2.1 Задание 1. Последовательность и порядок в List

Сохранить последовательность регистраций на курсы, поддержать вставку и удаление элементов, найти дубликаты. Сравнить ArrayList и LinkedList по скорости доступа по индексу и обходу.

Для сохранения порядка регистраций выбран ArrayList<CourseCode>, обеспечивающий быстрый доступ по индексу $O(1)$ и высокую локальность данных в памяти. Для поиска дубликатов с сохранением порядка первого повтора реализован метод DuplicateFinder.duplicates, работающий за $O(N)$ по времени с использованием HashSet (для отслеживания уже встреченных элементов) и LinkedHashSet (для сохранения порядка дубликатов).

Листинг 1 — Поиск дубликатов с сохранением порядка

```
public class DuplicateFinder {
    public static List<CourseCode> duplicates(List<CourseCode> codes) {
        Set<CourseCode> seen = new HashSet<>();
        Set<CourseCode> duplicates = new LinkedHashSet<>();
        for (CourseCode code : codes) {
            if (!seen.add(code)) {
                duplicates.add(code);
            }
        }
        return List.copyOf(duplicates);
    }
}
```

Эксперимент в ListPerformanceDemo (100 000 элементов, 200 000 случайных чтений) показал:

- Случайный доступ по индексу: ArrayList производит за $O(1)$ время, тогда как LinkedList требует $O(N)$ операций на перемещение по указателям узлов, из-за чего разница в скорости составляет несколько порядков.
- Последовательный обход: ArrayList опережает LinkedList за счет непрерывного расположения элементов в памяти и эффективного использования кэша процессора.

2.2 Задание 2. Гарантия уникальности и Set

Удалить дубликаты кодов курсов CourseCode, сохранив порядок их первого появления. Объяснить роль методов equals и hashCode.

Использован LinkedHashSet, который гарантирует уникальность элементов как HashSet, но при этом сохраняет порядок вставки элементов через двусвязный список.

Листинг 2 — Удаление дубликатов через LinkedHashSet

```
public final class Deduplicator {
    public static List<CourseCode> unique(List<CourseCode> codes) {
        Set<CourseCode> set = new LinkedHashSet<>();
        for (CourseCode code : codes) {
            if (code == null) throw new NullPointerException("code must
not be null");
            set.add(code);
        }
        return List.copyOf(set);
    }
}
```

Set определяет дубликаты в два шага: 1) Вычисляется hashCode() объекта для определения индекса корзины (bucket); 2) Внутри корзины объекты сравниваются через equals(). Если у класса не переопределены equals и hashCode (как в тестовом классе PlainCode), сравнение происходит по ссылкам (identity), и идентичные по содержимому объекты воспринимаются как разные. В CourseCode равенство базируется на нормализованной строке, благодаря чему "java-101", " JAVA_101 " и "Java 101" корректно схлопываются в один элемент.

2.3 Задание 3. Индексация данных и Map

Построить индекс курсов CourseId -> Course. Обработать коллизии при повторном ID без молчаливой перезаписи. Сравнить ключевые методы Map.

Использован LinkedHashMap<CourseId, Course>, сохраняющий порядок добавления. Добавление выполняется через putIfAbsent: если метод возвращает не null, значит курс с таким идентификатором уже существует, и генерируется IllegalStateException.

Листинг 3 — Индексация данных с защитой от дубликатов

```
public class CourseIndex {
    private final Map<CourseId, Course> byId = new LinkedHashMap<>();
    public void add(Course course) {
        if (course == null) throw new NullPointerException("course must
not be null");
        Course previous = byId.putIfAbsent(course.id(), course);
        if (previous != null) {
            throw new IllegalStateException("duplicate id: " +
course.id());
        }
    }
    public Course findOrAdd(CourseId id, Supplier<Course> factory) {
        if (id == null) throw new NullPointerException("id must not be
null");
    }
}
```

```

        return byId.computeIfAbsent(id, key -> factory.get());
    }
}

```

Сравнение методов Map: `get(k)` возвращает значение или `null` (не позволяет отличить отсутствие ключа от значения `null`); `containsKey(k)` проверяет наличие ключа, но вызов перед `get` приводит к двойному поиску; `putIfAbsent(k, v)` атомарно добавляет значение, только если ключ отсутствует; `computeIfAbsent(k, f)` вычисляет значение через фабрику только при промахе кэша, исключая лишнее создание объектов.

2.4 Задание 4. Организация очередей (Queue)

Реализовать очередь ожидания записи на курс по принципу FIFO. Сравнить мягкие и строгие методы Queue. В качестве реализации использован `ArrayDeque<Long>`, превосходящий `LinkedList` по быстродействию и памяти за счет кольцевого буфера.

Таблица 1 — Разграничение методов интерфейса Queue

Операция	Мягкий метод (сигнал через false/null)	Строгий метод (генерация исключения)	Поведение при пустой/полной очереди
Добавление	<code>offer(e)</code>	<code>add(e)</code>	<code>offer -> false</code> , <code>add -> IllegalStateException</code>
Просмотр головы	<code>peek()</code>	<code>element()</code>	<code>peek -> null</code> , <code>element -> NoSuchElementException</code>
Извлечение головы	<code>poll()</code>	<code>remove()</code>	<code>poll -> null</code> , <code>remove -> NoSuchElementException</code>

2.5 Задание 5. Безопасность модификации коллекций при обходе

Воспроизвести `ConcurrentModificationException` при удалении элементов из списка в цикле `for-each` и продемонстрировать способы его устранения.

Причина: цикл `for-each` использует `Iterator`, а коллекция отслеживает `modCount`. Прямой вызов `list.remove()` рассогласовывает `modCount` и `expectedModCount` итератора, вызывая исключение при следующем `iteration.next()`.

Листинг 4 — Безопасное удаление через `removeIf`

```
public static void removeWithRemoveIf(List<CourseCode> codes, CourseCode
target) {
    codes.removeIf(code -> code.equals(target));
}
```

3. Обобщенное программирование (Generics)

3.1 Задание 6. Обобщенные типы данных (Result<T> и Page<T>)

Разработать обобщенный класс результатов операций Result<T> и неизменяемую страницу выдачи Page<T> без использования raw types. Result<T> представляет собой монадический контейнер успеха или ошибки с проверкой инварианта в приватном конструкторе.

Листинг 5 — Обобщенный класс Result<T>

```
public final class Result<T> {
    private final T value;
    private final String error;
    private Result(T value, String error) {
        if ((value == null) == (error == null)) {
            throw new IllegalArgumentException("exactly one of
value/error must be set");
        }
        this.value = value;
        this.error = error;
    }
    public static <T> Result<T> ok(T value) {
        if (value == null) throw new NullPointerException("value must not
be null");
        return new Result<>(value, null);
    }
    public static <T> Result<T> error(String message) {
        if (message == null || message.isBlank()) throw new
IllegalArgumentException("message must not be blank");
        return new Result<>(null, message);
    }
    public <R> Result<R> map(Function<? super T, ? extends R> mapper) {
        if (isError()) return error(error);
        return ok(mapper.apply(value));
    }
}
```

3.2 Задание 7. Обобщенные методы и утилиты

Реализовать универсальные обобщенные методы firstOrThrow, swap и indexBy с обработкой конфликтов ключей. Метод indexBy преобразует список элементов в Map<K, T>, кидая IllegalStateException при совпадении ключей.

Листинг 6 — Метод индексации списков indexBy

```
public final class GenericLists {
```

```

    public static <T, K> Map<K, T> indexBy(List<T> items, Function<T, K>
keyFn) {
        Map<K, T> index = new LinkedHashMap<>();
        for (T item : items) {
            if (item == null) throw new NullPointerException("item must
not be null");
            K key = keyFn.apply(item);
            T previous = index.putIfAbsent(key, item);
            if (previous != null) {
                throw new IllegalStateException("duplicate key " + key +
": " + previous + " and " + item);
            }
        }
        return index;
    }
}

```

3.3 Задание 8. Ограничения типов и Bounded Wildcards

Реализовать поиск максимального элемента в списке с границей `<T extends Comparable<? super T>>`. Запись `? super T` критически важна для подклассов, наследующих `compareTo` от базового класса.

Листинг 7 — Поиск максимума с Bounded Wildcard

```

public static <T extends Comparable<? super T>> T max(List<? extends T>
items) {
    if (items == null) throw new NullPointerException("items must not be
null");
    if (items.isEmpty()) throw new NoSuchElementException("items is
empty");
    T best = items.get(0);
    for (T item : items) {
        if (item.compareTo(best) > 0) {
            best = item;
        }
    }
    return best;
}

```

3.4 Задание 9. Практическое применение принципа PECS

Продемонстрировать правило PECS (Producer Extends, Consumer Super) на примере копирования коллекций: `Producer Extends` (`? extends T`) только читается, `Consumer Super` (`? super T`) только наполняется.

Листинг 8 — Копирование согласно принципу PECS

```
public final class CollectionCopies {  
    public static <T> void copy(List<? extends T> source, List<? super T>  
target) {  
        for (T item : source) {  
            target.add(item);  
        }  
    }  
}
```

4. Контракты структур данных и архитектурное проектирование

4.1 Задание 10. Соблюдение контракта ключа в хэш-таблицах

Исследовать влияние мутабельности полей в equals/hashCode. В BadCourse изменение popularity после помещения в HashSet меняет hashCode, из-за чего объект теряется в старой корзине (contains -> false, get -> null, a size по-прежнему 1).

4.2 Задание 11. Выбор оптимальной структуры данных под нагрузку

Таблица 2 — Выбор структур данных под требования нагрузки

№	Сценарий применения	Интерфейс и реализация	Временная сложность	Обоснование выбора
1	Каталог курсов по ID	Map -> HashMap	$O(1)$	Поиск и добавление за константное время.
2	Отсортированный Leaderboard	NavigableMap -> TreeMap	$O(\log N)$	Автоматическая сортировка по ключу (баллу).
3	Очередь ожидания записи	Queue -> ArrayDeque	$O(1)$ на концах	FIFO порядок, минимальные накладные расходы.
4	Уникальные теги с сохранением порядка	Set -> LinkedHashSet	$O(1)$	Уникальность + порядок вставки.
5	Диапазонный поиск ID [from, to]	NavigableMap -> TreeMap	$O(\log N + K)$	Выборка поддеревы через subMap.

№	Сценарий применения	Интерфейс и реализация	Временная сложность	Обоснование выбора
6	Доступ к урокам по индексу	List -> ArrayList	O(1)	Мгновенный доступ к элементу по индексу.

4.3 Задание 12. Проектирование обобщенного репозитория (Repository)

Спроектировать контракт Repository<ID, T> и реализацию InMemoryRepository<ID, T> с защитой инвариантов (upsert-семантика, List.copyOf для снимка состояния).

Листинг 9 — Реализация InMemoryRepository

```
public class InMemoryRepository<ID, T> implements Repository<ID, T> {
    private final Map<ID, T> store = new LinkedHashMap<>();
    private final Function<T, ID> idFn;
    public InMemoryRepository(Function<T, ID> idFn) {
        if (idFn == null) throw new NullPointerException("idFn must not be null");
        this.idFn = idFn;
    }
    @Override
    public void save(T entity) {
        if (entity == null) throw new NullPointerException("entity must not be null");
        ID id = idFn.apply(entity);
        if (id == null) throw new NullPointerException("id must not be null");
        store.put(id, entity);
    }
    @Override public List<T> findAll() { return List.copyOf(store.values()); }
    @Override public Optional<T> findById(ID id) { return Optional.ofNullable(store.get(id)); }
    @Override public boolean deleteById(ID id) { return store.remove(id) != null; }
}
```

4.4 Задание 13. Построение инвертированного индекса

Реализовать инвертированный индекс Map<Tag, Set<CourseId>> с операциями пересечения/объединения и автоматическим удалением пустых множеств.

Листинг 10 — Пересечение тегов в InvertedIndex

```
public Set<CourseId> coursesWithAll(Set<Tag> tags) {
    if (tags.isEmpty()) return Set.of();
    Set<CourseId> result = null;
    for (Tag tag : tags) {
        Set<CourseId> ids = index.get(tag);
        if (ids == null) return Set.of();
        if (result == null) result = new HashSet<>(ids);
        else result.retainAll(ids);
    }
    return Set.copyOf(result);
}
```

4.5 Задание 14. Реализация LRU-кэша на базе LinkedHashMap

Разработать обобщенный LRU-кэш на базе LinkedHashMap в режиме `accessOrder=true` и с переопределенным `removeEldestEntry`.

Листинг 11 — Конструктор LruCache

```
public LruCache(int capacity) {
    if (capacity <= 0) throw new IllegalArgumentException("capacity must be > 0");
    this.capacity = capacity;
    this.map = new LinkedHashMap<>(capacity, 0.75f, true) {
        @Override
        protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
            return size() > LruCache.this.capacity;
        }
    };
}
```

4.6 Задание 15. Построение и анализ графа пререквизитов

Топологическая сортировка графа по алгоритму Кана за $O(V + E)$ и поиск циклов через three-color DFS с генерацией подробного пути цикла.

Листинг 12 — Топологическая сортировка графа

```
public List<V> order() {
    Map<V, Integer> indegree = new HashMap<>();
    for (V v : insertionOrder) indegree.put(v, 0);
    for (Map.Entry<V, Set<V>> entry : prereqOf.entrySet())
        indegree.put(entry.getKey(), entry.getValue().size());
    Deque<V> ready = new ArrayDeque<>();
    for (V v : insertionOrder) if (indegree.get(v) == 0) ready.add(v);
    List<V> result = new ArrayList<>();
```

```
while (!ready.isEmpty()) {
    V v = ready.poll(); result.add(v);
    for (V dependent : dependents.getOrDefault(v, Set.of())) {
        int left = indegree.get(dependent) - 1;
indegree.put(dependent, left);
        if (left == 0) ready.add(dependent);
    }
}
if (result.size() != insertionOrder.size()) throw new
IllegalStateException("cycle detected: " +
findCycle().orElse(List.of()));
return List.copyOf(result);
}
```

5. Заключение

В ходе выполнения практической работы № 5 были успешно освоены ключевые концепции работы с коллекциями и обобщенным программированием в Java:

1. Проведен сравнительный анализ асимптотической сложности и реальной производительности основных реализаций списков, множеств, отображений и очередей.
2. Изучены механизмы работы итераторов, причины возникновения `ConcurrentModificationException` и правильные паттерны модификации коллекций во время обхода.
3. Освоено применение Generics, обобщенных типов и методов, а также принципа PECS (Producer Extends, Consumer Super) для создания гибких и типобезопасных API.
4. Продемонстрирована критическая важность соблюдения контракта `equals/hashCode` для корректной работы хэш-таблиц.
5. На практике спроектированы высокоуровневые структуры данных: LRU-кэш, инвертированный индекс, обобщенный репозиторий и граф зависимостей с алгоритмом Кана и поиском циклов.

Приложение А. Ключевые листинги программы

Листинг 13 — CourseRegistry.java (задание 1)

```
import src.CourseCode;
import java.util.ArrayList;
import java.util.List;
public class CourseRegistry {
    private final List<CourseCode> registrations = new ArrayList<>();
    public void register(CourseCode code) {
        if (code == null) throw new NullPointerException("code must not
be null");
        registrations.add(code);
    }
    public void insertAt(int index, CourseCode code) {
        registrations.add(index, code);
    }
    public boolean remove(CourseCode code) {
        return registrations.remove(code);
    }
    public CourseCode at(int index) {
        return registrations.get(index);
    }
    public List<CourseCode> sequence() {
        return List.copyOf(registrations);
    }
    public int size() {
        return registrations.size();
    }
}
```

Листинг 14 — DuplicateFinder.java (задание 1)

```
import src.CourseCode;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;
public class DuplicateFinder {
    public static List<CourseCode> duplicates(List<CourseCode> codes) {
        Set<CourseCode> seen = new HashSet<>();
        Set<CourseCode> duplicates = new LinkedHashSet<>();
        for (CourseCode code : codes) {
            if (!seen.add(code)) {
                duplicates.add(code);
            }
        }
        return List.copyOf(duplicates);
    }
}
```

```
}
```

Листинг 15 — ListPerformanceDemo.java (задание 1)

```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Random;
public class ListPerformanceDemo {
    private static final int N = 100_000;
    private static final int READS = 200_000;
    public static void main(String[] args) {
        List<Integer> arrayList = fill(new ArrayList<>());
        List<Integer> linkedList = fill(new LinkedList<>());
        System.out.println("random access: arrayList=" +
randomAccess(arrayList) + " ms, linkedList=" + randomAccess(linkedList) +
" ms");
        System.out.println("traversal: arrayList=" + traversal(arrayList)
+ " ms, linkedList=" + traversal(linkedList) + " ms");
    }
    static List<Integer> fill(List<Integer> list) {
        for (int i = 0; i < N; i++) list.add(i);
        return list;
    }
    static long randomAccess(List<Integer> list) {
        Random random = new Random(1);
        long sum = 0;
        long start = System.nanoTime();
        for (int i = 0; i < READS; i++) {
            sum += list.get(random.nextInt(N));
        }
        long elapsed = System.nanoTime() - start;
        if (sum == Long.MIN_VALUE) System.out.print("");
        return elapsed / 1_000_000;
    }
    static long traversal(List<Integer> list) {
        long sum = 0;
        long start = System.nanoTime();
        for (int value : list) {
            sum += value;
        }
        long elapsed = System.nanoTime() - start;
        if (sum == Long.MIN_VALUE) System.out.print("");
        return elapsed / 1_000_000;
    }
}
```

Листинг 16 — CourseCode.java (задания 1, 2, 5)

```

package src;
import java.util.Locale;
public final class CourseCode {
    public static final int MIN_LENGTH = 3;
    public static final int MAX_LENGTH = 10;
    private final String value;
    private CourseCode(String value) {
        this.value = value;
    }
    public static CourseCode parse(String raw) {
        String normalized = normalize(raw);
        if (normalized.isEmpty()) {
            throw new IllegalArgumentException("code must not be blank");
        }
        if (normalized.length() < MIN_LENGTH || normalized.length() >
MAX_LENGTH) {
            throw new IllegalArgumentException("code length must be in ["
+ MIN_LENGTH + ", " + MAX_LENGTH + "]");
        }
        for (int i = 0; i < normalized.length(); i++) {
            char c = normalized.charAt(i);
            boolean latinLetterOrDigit = (c >= 'A' && c <= 'Z') || (c >=
'0' && c <= '9');
            if (!latinLetterOrDigit) {
                throw new IllegalArgumentException("code must contain
only latin letters and digits, got '" + c + "'");
            }
        }
        return new CourseCode(normalized);
    }
    public static String normalize(String raw) {
        return raw.trim().toUpperCase(Locale.ROOT).replace(" ",
"").replace("-", "").replace("_", "");
    }
    public String value() {
        return value;
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof CourseCode other)) return false;
        return value.equals(other.value);
    }
    @Override
    public int hashCode() {
        return value.hashCode();
    }
    @Override

```

```

    public String toString() {
        return value;
    }
}

```

Листинг 17 — Deduplicator.java (задание 2)

```

import src.CourseCode;
import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;
public final class Deduplicator {
    private Deduplicator() {}
    public static List<CourseCode> unique(List<CourseCode> codes) {
        Set<CourseCode> set = new LinkedHashSet<>();
        for (CourseCode code : codes) {
            if (code == null) throw new NullPointerException("code must
not be null");
            set.add(code);
        }
        return List.copyOf(set);
    }
    public static List<CourseCode> uniqueNaive(List<CourseCode> codes) {
        List<CourseCode> result = new ArrayList<>();
        for (CourseCode code : codes) {
            if (!result.contains(code)) {
                result.add(code);
            }
        }
        return result;
    }
}

```

Листинг 18 — CourseIndex.java (задание 3)

```

import src.Course;
import src.CourseId;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Optional;
import java.util.function.Supplier;
public class CourseIndex {
    private final Map<CourseId, Course> byId = new LinkedHashMap<>();
    public void add(Course course) {
        if (course == null) throw new NullPointerException("course must
not be null");
        Course previous = byId.putIfAbsent(course.id(), course);
    }
}

```

```

        if (previous != null) {
            throw new IllegalStateException("duplicate id: " +
course.id());
        }
    }
    public void replace(Course course) {
        if (course == null) throw new NullPointerException("course must
not be null");
        byId.put(course.id(), course);
    }
    public Optional<Course> find(CourseId id) {
        return Optional.ofNullable(byId.get(id));
    }
    public boolean contains(CourseId id) {
        return byId.containsKey(id);
    }
    public Course findOrAdd(CourseId id, Supplier<Course> factory) {
        if (id == null) throw new NullPointerException("id must not be
null");
        return byId.computeIfAbsent(id, key -> factory.get());
    }
    public int size() {
        return byId.size();
    }
}

```

Листинг 19 — Course.java (задания 3, 6, 9, 10, 12, 13)

```

package src;
import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.Objects;
import java.util.Set;
public class Course {
    private final CourseId id;
    private final String title;
    private final int durationHours;
    private int popularity;
    private final Set<Tag> tags = new LinkedHashSet<>();
    public Course(CourseId id, String title, int durationHours) {
        if (id == null) throw new NullPointerException("id must not be
null");
        if (title == null || title.isBlank()) throw new
IllegalArgumentException("title must not be blank");
        if (durationHours <= 0) throw new
IllegalArgumentException("durationHours must be > 0");
        this.id = id;
        this.title = title.trim();
    }
}

```

```

        this.durationHours = durationHours;
    }
    public CourseId id() { return id; }
    public String title() { return title; }
    public int durationHours() { return durationHours; }
    public int popularity() { return popularity; }
    public void setPopularity(int popularity) { this.popularity =
popularity; }
    public void addTag(Tag tag) {
        if (tag == null) throw new NullPointerException("tag must not be
null");
        tags.add(tag);
    }
    public Set<Tag> tags() { return Collections.unmodifiableSet(tags); }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Course other)) return false;
        return id.equals(other.id);
    }
    @Override
    public int hashCode() { return Objects.hash(id); }
    @Override
    public String toString() { return "Course[" + id + ", " + title +
"]"; }
}

```

Листинг 20 — CourseId.java (задания 3, 6, 8, 10, 11, 12, 13)

```

package src;
import java.util.Objects;
public final class CourseId implements Comparable<CourseId> {
    private final long value;
    public CourseId(long value) {
        if (value <= 0) throw new IllegalArgumentException("id must be >
0");
        this.value = value;
    }
    public long value() { return value; }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof CourseId other)) return false;
        return value == other.value;
    }
    @Override
    public int hashCode() { return Objects.hash(value); }
    @Override

```

```

    public int compareTo(CourseId other) { return Long.compare(value,
other.value); }
    @Override
    public String toString() { return "CourseId(" + value + ")"; }
}

```

Листинг 21 — Waitlist.java (задание 4)

```

import java.util.ArrayDeque;
import java.util.Optional;
import java.util.Queue;
public class Waitlist {
    private final Queue<Long> queue = new ArrayDeque<>();
    public boolean enqueue(long studentId) { return
queue.offer(studentId); }
    public Long nextOrNull() { return queue.peek(); }
    public Long takeOrNull() { return queue.poll(); }
    public void enqueueOrThrow(long studentId) { queue.add(studentId); }
    public long nextOrThrow() { return queue.element(); }
    public long takeOrThrow() { return queue.remove(); }
    public Optional<Long> next() { return
Optional.ofNullable(queue.peek()); }
    public Optional<Long> take() { return
Optional.ofNullable(queue.poll()); }
    public int size() { return queue.size(); }
}

```

Листинг 22 — SafeRemoval.java (задание 5)

```

import src.CourseCode;
import java.util.Iterator;
import java.util.List;
public final class SafeRemoval {
    private SafeRemoval() {}
    public static void removeWithIterator(List<CourseCode> codes,
CourseCode target) {
        Iterator<CourseCode> iterator = codes.iterator();
        while (iterator.hasNext()) {
            if (iterator.next().equals(target)) {
                iterator.remove();
            }
        }
    }
    public static void removeWithRemoveIf(List<CourseCode> codes,
CourseCode target) {
        codes.removeIf(code -> code.equals(target));
    }
}

```

Листинг 23 — Result.java (задание 6)

```
package src;
import java.util.Objects;
import java.util.Optional;
import java.util.function.Function;
public final class Result<T> {
    private final T value;
    private final String error;
    private Result(T value, String error) {
        if ((value == null) == (error == null)) {
            throw new IllegalArgumentException("exactly one of
value/error must be set");
        }
        this.value = value;
        this.error = error;
    }
    public static <T> Result<T> ok(T value) {
        if (value == null) throw new NullPointerException("value must not
be null");
        return new Result<>(value, null);
    }
    public static <T> Result<T> error(String message) {
        if (message == null || message.isBlank()) throw new
IllegalArgumentException("message must not be blank");
        return new Result<>(null, message);
    }
    public boolean isOk() { return error == null; }
    public boolean isError() { return error != null; }
    public T value() {
        if (isError()) throw new IllegalStateException("no value: " +
error);
        return value;
    }
    public String error() {
        if (isOk()) throw new IllegalStateException("result is ok, no
error");
        return error;
    }
    public Optional<T> toOptional() { return Optional.ofNullable(value);
}
    public <R> Result<R> map(Function<? super T, ? extends R> mapper) {
        if (isError()) return error(error);
        return ok(mapper.apply(value));
    }
    public <R> Result<R> flatMap(Function<? super T, Result<R>> mapper) {
        if (isError()) return error(error);
        return mapper.apply(value);
    }
}
```

```

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Result<?> other)) return false;
    return Objects.equals(value, other.value) &&
Objects.equals(error, other.error);
}
@Override
public int hashCode() { return Objects.hash(value, error); }
@Override
public String toString() { return isOk() ? "Ok(" + value + ")" :
"Error(" + error + ")"; }
}

```

Листинг 24 — Page.java (задание 6)

```

package src;
import java.util.List;
import java.util.Objects;
public final class Page<T> {
    private final List<T> items;
    private final int pageNumber;
    private final int pageSize;
    private final long totalItems;
    public Page(List<T> items, int pageNumber, int pageSize, long
totalItems) {
        if (items == null) throw new NullPointerException("items must not
be null");
        if (pageNumber < 0) throw new
IllegalArgumentException("pageNumber must be >= 0");
        if (pageSize <= 0) throw new IllegalArgumentException("pageSize
must be > 0");
        if (totalItems < items.size()) throw new
IllegalArgumentException("totalItems cannot be less than items on the
page");
        this.items = List.copyOf(items);
        this.pageNumber = pageNumber;
        this.pageSize = pageSize;
        this.totalItems = totalItems;
    }
    public List<T> items() { return items; }
    public int pageNumber() { return pageNumber; }
    public int pageSize() { return pageSize; }
    public long totalItems() { return totalItems; }
    public int totalPages() { return (int) ((totalItems + pageSize - 1) /
pageSize); }
    public boolean hasNext() { return pageNumber + 1 < totalPages(); }
    public boolean isEmpty() { return items.isEmpty(); }
}

```

```

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Page<?> other)) return false;
    return pageNumber == other.pageNumber && pageSize ==
other.pageSize && totalItems == other.totalItems &&
items.equals(other.items);
}
@Override
public int hashCode() { return Objects.hash(items, pageNumber,
pageSize, totalItems); }
@Override
public String toString() { return "Page[" + pageNumber + "/" +
totalPages() + ", items=" + items.size() + "]; }
}

```

Листинг 25 — GenericLists.java (задание 7)

```

package src;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.BinaryOperator;
import java.util.function.Function;
public final class GenericLists {
    private GenericLists() {}
    public static <T> T firstOrThrow(List<T> list) {
        if (list == null) throw new NullPointerException("list must not
be null");
        if (list.isEmpty()) throw new
java.util.NoSuchElementException("list is empty");
        return list.get(0);
    }
    public static <T> void swap(List<T> list, int i, int j) {
        if (list == null) throw new NullPointerException("list must not
be null");
        if (i < 0 || j < 0 || i >= list.size() || j >= list.size()) {
            throw new IndexOutOfBoundsException("index out of range: " +
i + ", " + j);
        }
        T tmp = list.get(i);
        list.set(i, list.get(j));
        list.set(j, tmp);
    }
    public static <T, K> Map<K, T> indexBy(List<T> items, Function<T, K>
keyFn) {
        Map<K, T> index = new LinkedHashMap<>();
        for (T item : items) {

```

```

        if (item == null) throw new NullPointerException("item must
not be null");
        K key = keyFn.apply(item);
        if (key == null) throw new NullPointerException("key must not
be null");
        T previous = index.putIfAbsent(key, item);
        if (previous != null) {
            throw new IllegalStateException("duplicate key " + key +
": " + previous + " and " + item);
        }
    }
    return index;
}

public static <T, K> Map<K, T> indexBy(List<T> items, Function<T, K>
keyFn, BinaryOperator<T> merge) {
    Map<K, T> index = new LinkedHashMap<>();
    for (T item : items) {
        if (item == null) throw new NullPointerException("item must
not be null");
        index.merge(keyFn.apply(item), item, merge);
    }
    return index;
}
}

```

Листинг 26 — Comparables.java (задание 8)

```

package src;
import java.util.List;
import java.util.NoSuchElementException;
public final class Comparables {
    private Comparables() {}
    public static <T extends Comparable<? super T>> T max(List<? extends
T> items) {
        if (items == null) throw new NullPointerException("items must not
be null");
        if (items.isEmpty()) throw new NoSuchElementException("items is
empty");
        T best = items.get(0);
        for (T item : items) {
            if (item.compareTo(best) > 0) {
                best = item;
            }
        }
        return best;
    }
    public static <T extends Comparable<? super T>> T min(List<? extends
T> items) {

```

```

        if (items == null) throw new NullPointerException("items must not
be null");
        if (items.isEmpty()) throw new NoSuchElementException("items is
empty");
        T best = items.get(0);
        for (T item : items) {
            if (item.compareTo(best) < 0) {
                best = item;
            }
        }
        return best;
    }
}

```

Листинг 27 — CollectionCopies.java (задание 9)

```

package src;
import java.util.List;
public final class CollectionCopies {
    private CollectionCopies() {}
    public static <T> void copy(List<? extends T> source, List<? super T>
target) {
        if (source == null || target == null) throw new
NullPointerException("lists must not be null");
        for (T item : source) {
            target.add(item);
        }
    }
    public static void copyRaw(List<? extends Object> source, List<?
super Object> target) {
        for (Object item : source) {
            target.add(item);
        }
    }
}

```

Листинг 28 — BadCourse.java (задание 10)

```

import java.util.Objects;
public class BadCourse {
    private final long id;
    private int popularity;
    public BadCourse(long id, int popularity) {
        this.id = id;
        this.popularity = popularity;
    }
    public void setPopularity(int popularity) {
        this.popularity = popularity;
    }
}

```

```

    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof BadCourse other)) return false;
        return id == other.id && popularity == other.popularity;
    }
    @Override
    public int hashCode() {
        return Objects.hash(id, popularity);
    }
}

```

Листинг 29 — Leaderboard.java (задание 11)

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.NavigableMap;
import java.util.TreeMap;
public class Leaderboard {
    private final NavigableMap<Integer, List<String>> byScore = new
    TreeMap<>(Collections.reverseOrder());
    public void score(String student, int points) {
        byScore.computeIfAbsent(points, k -> new
    ArrayList<>()).add(student);
    }
    public List<String> top(int n) {
        List<String> result = new ArrayList<>();
        for (Map.Entry<Integer, List<String>> entry : byScore.entrySet())
        {
            for (String student : entry.getValue()) {
                if (result.size() == n) return List.copyOf(result);
                result.add(student);
            }
        }
        return List.copyOf(result);
    }
    public int bestScore() {
        return byScore.isEmpty() ? -1 : byScore.firstKey();
    }
    public Map<Integer, List<String>> snapshot() {
        return new LinkedHashMap<>(byScore);
    }
}

```

Листинг 30 — RangeIndex.java (задание 11)

```
import src.CourseId;
import java.util.NavigableMap;
import java.util.SortedMap;
import java.util.TreeMap;
public class RangeIndex {
    private final NavigableMap<CourseId, String> byId = new TreeMap<>();
    public void put(CourseId id, String title) {
        byId.put(id, title);
    }
    public SortedMap<CourseId, String> range(long from, long to) {
        if (from > to) throw new IllegalArgumentException("from must be
<= to");
        return byId.subMap(new CourseId(from), true, new CourseId(to),
true);
    }
    public int size() {
        return byId.size();
    }
}
```

Листинг 31 — Repository.java (задание 12)

```
package src;
import java.util.List;
import java.util.Optional;
public interface Repository<ID, T> {
    void save(T entity);
    Optional<T> findById(ID id);
    List<T> findAll();
    boolean deleteById(ID id);
}
```

Листинг 32 — InMemoryRepository.java (задание 12)

```
package src;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.function.Function;
public class InMemoryRepository<ID, T> implements Repository<ID, T> {
    private final Map<ID, T> store = new LinkedHashMap<>();
    private final Function<T, ID> idFn;
    public InMemoryRepository(Function<T, ID> idFn) {
        if (idFn == null) throw new NullPointerException("idFn must not
be null");
        this.idFn = idFn;
    }
}
```

```

    }
    @Override
    public void save(T entity) {
        if (entity == null) throw new NullPointerException("entity must
not be null");
        ID id = idFn.apply(entity);
        if (id == null) throw new NullPointerException("id must not be
null");
        store.put(id, entity);
    }
    @Override
    public Optional<T> findById(ID id) {
        if (id == null) throw new NullPointerException("id must not be
null");
        return Optional.ofNullable(store.get(id));
    }
    @Override
    public List<T> findAll() {
        return List.copyOf(store.values());
    }
    @Override
    public boolean deleteById(ID id) {
        if (id == null) throw new NullPointerException("id must not be
null");
        return store.remove(id) != null;
    }
    public int size() {
        return store.size();
    }
}

```

Листинг 33 — InvertedIndex.java (задание 13)

```

import src.Course;
import src.CourseId;
import src.Tag;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Map;
import java.util.Set;
public class InvertedIndex {
    private final Map<Tag, Set<CourseId>> index = new HashMap<>();
    public void add(Course course) {
        if (course == null) throw new NullPointerException("course must
not be null");
        for (Tag tag : course.tags()) {

```

```

        index.computeIfAbsent(tag, k -> new
LinkedHashSet<>()).add(course.id());
    }
}
public void remove(CourseId id, Set<Tag> tags) {
    if (id == null || tags == null) throw new
NullPointerException("id and tags must not be null");
    for (Tag tag : tags) {
        Set<CourseId> ids = index.get(tag);
        if (ids == null) continue;
        ids.remove(id);
        if (ids.isEmpty()) {
            index.remove(tag);
        }
    }
}
public Set<CourseId> coursesWithAll(Set<Tag> tags) {
    if (tags.isEmpty()) return Set.of();
    Set<CourseId> result = null;
    for (Tag tag : tags) {
        Set<CourseId> ids = index.get(tag);
        if (ids == null) return Set.of();
        if (result == null) {
            result = new HashSet<>(ids);
        } else {
            result.retainAll(ids);
        }
    }
    return Set.copyOf(result);
}
public Set<CourseId> coursesWithAny(Set<Tag> tags) {
    Set<CourseId> result = new HashSet<>();
    for (Tag tag : tags) {
        Set<CourseId> ids = index.get(tag);
        if (ids != null) {
            result.addAll(ids);
        }
    }
    return Set.copyOf(result);
}
public int tagCount() {
    return index.size();
}
}

```

Листинг 34 — Tag.java (задание 13)

```
package src;
```

```

import java.util.Locale;
public final class Tag implements Comparable<Tag> {
    private final String name;
    private Tag(String name) {
        this.name = name;
    }
    public static Tag of(String raw) {
        if (raw == null) throw new NullPointerException("tag must not be
null");
        String normalized = raw.trim().toLowerCase(Locale.ROOT);
        if (normalized.isEmpty()) {
            throw new IllegalArgumentException("tag must not be blank");
        }
        return new Tag(normalized);
    }
    public String name() {
        return name;
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Tag other)) return false;
        return name.equals(other.name);
    }
    @Override
    public int hashCode() {
        return name.hashCode();
    }
    @Override
    public int compareTo(Tag other) {
        return name.compareTo(other.name);
    }
    @Override
    public String toString() {
        return "#" + name;
    }
}

```

Листинг 35 — LruCache.java (задание 14)

```

package src;
import java.util.LinkedHashMap;
import java.util.Map;
public class LruCache<K, V> {
    private final int capacity;
    private final Map<K, V> map;
    public LruCache(int capacity) {

```

```

        if (capacity <= 0) throw new IllegalArgumentException("capacity
must be > 0");
        this.capacity = capacity;
        this.map = new LinkedHashMap<>(capacity, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
                return size() > LruCache.this.capacity;
            }
        };
    }
    public V get(K key) {
        if (key == null) throw new NullPointerException("key must not be
null");
        return map.get(key);
    }
    public void put(K key, V value) {
        if (key == null) throw new NullPointerException("key must not be
null");
        if (value == null) throw new NullPointerException("value must not
be null");
        map.put(key, value);
    }
    public boolean containsKey(K key) {
        if (key == null) throw new NullPointerException("key must not be
null");
        return map.containsKey(key);
    }
    public int size() {
        return map.size();
    }
    public void clear() {
        map.clear();
    }
}

```

Листинг 36 — PrerequisiteGraph.java (задание 15)

```

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.HashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
public class PrerequisiteGraph<V> {
    private final Map<V, Set<V>> prereqOf = new HashMap<>();
    private final Map<V, Set<V>> dependents = new HashMap<>();
    private final List<V> insertionOrder = new java.util.ArrayList<>();
}

```

```

    public void register(V course) {
        if (course == null) throw new NullPointerException("course must
not be null");
        prereqOf.computeIfAbsent(course, k -> {
            insertionOrder.add(k);
            return new LinkedHashSet<>();
        });
        dependents.computeIfAbsent(course, k -> new LinkedHashSet<>());
    }
    public void addPrerequisite(V course, V prereq) {
        register(course);
        register(prereq);
        prereqOf.get(course).add(prereq);
        dependents.get(prereq).add(course);
    }
    public boolean contains(V course) {
        return prereqOf.containsKey(course);
    }
    public int vertexCount() {
        return insertionOrder.size();
    }
    public List<V> order() {
        Map<V, Integer> indegree = new HashMap<>();
        for (V v : insertionOrder) indegree.put(v, 0);
        for (Map.Entry<V, Set<V>> entry : prereqOf.entrySet()) {
            indegree.put(entry.getKey(), entry.getValue().size());
        }
        Deque<V> ready = new ArrayDeque<>();
        for (V v : insertionOrder) {
            if (indegree.get(v) == 0) ready.add(v);
        }
        List<V> result = new java.util.ArrayList<>();
        while (!ready.isEmpty()) {
            V v = ready.poll();
            result.add(v);
            for (V dependent : dependents.getOrDefault(v, Set.of())) {
                int left = indegree.get(dependent) - 1;
                indegree.put(dependent, left);
                if (left == 0) ready.add(dependent);
            }
        }
        if (result.size() != insertionOrder.size()) {
            throw new IllegalStateException("cycle detected: " +
findCycle().orElse(List.of()));
        }
        return List.copyOf(result);
    }
    public java.util.Optional<List<V>> findCycle() {

```

```

Map<V, Integer> color = new HashMap<>();
for (V v : insertionOrder) color.put(v, 0);
for (V start : insertionOrder) {
    if (color.get(start) != 0) continue;
    Deque<V> stack = new ArrayDeque<>();
    Deque<java.util.Iterator<V>> iterators = new ArrayDeque<>();
    stack.push(start);
    color.put(start, 1);
    iterators.push(prereqOf.getOrDefault(start,
Set.of()).iterator());
    while (!stack.isEmpty()) {
        V v = stack.peek();
        java.util.Iterator<V> it = iterators.peek();
        if (it.hasNext()) {
            V next = it.next();
            int nextColor = color.getOrDefault(next, 0);
            if (nextColor == 1) {
                List<V> path = new java.util.ArrayList<>(stack);
                java.util.Collections.reverse(path);
                int from = path.indexOf(next);
                List<V> cycle = new
java.util.ArrayList<>(path.subList(from, path.size()));
                cycle.add(next);
                return java.util.Optional.of(cycle);
            }
            if (nextColor == 0) {
                color.put(next, 1);
                stack.push(next);
                iterators.push(prereqOf.getOrDefault(next,
Set.of()).iterator());
            }
        } else {
            color.put(v, 2);
            stack.pop();
            iterators.pop();
        }
    }
    return java.util.Optional.empty();
}
}

```

Листинг 37 — pom.xml (все задания)

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

```

```

<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
<artifactId>coursehub-collections</artifactId>
<version>1.0-SNAPSHOT</version>
<properties>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
</properties>
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
<build>
  <sourceDirectory>${project.basedir}/src</sourceDirectory>
<testSourceDirectory>${project.basedir}/${task.dir}</testSourceDirectory>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <configuration>
        <runOrder>random</runOrder>
      </configuration>
    </plugin>
  </plugins>
</build>
<profiles>
  <profile>
    <id>task1</id>
    <activation><activeByDefault>true</activeByDefault></activation>
    <properties><task.dir>task1</task.dir></properties>
  </profile>

  <profile><id>task2</id><properties><task.dir>task2</task.dir></properties>
</profile>

  <profile><id>task3</id><properties><task.dir>task3</task.dir></properties>
</profile>

  <profile><id>task4</id><properties><task.dir>task4</task.dir></properties>
</profile>

  <profile><id>task5</id><properties><task.dir>task5</task.dir></properties>
</profile>

```

```
<profile><id>task6</id><properties><task.dir>task6</task.dir></properties>
</profile>

<profile><id>task7</id><properties><task.dir>task7</task.dir></properties>
</profile>

<profile><id>task8</id><properties><task.dir>task8</task.dir></properties>
</profile>

<profile><id>task9</id><properties><task.dir>task9</task.dir></properties>
</profile>

<profile><id>task10</id><properties><task.dir>task10</task.dir></properties>
</profile>

<profile><id>task11</id><properties><task.dir>task11</task.dir></properties>
</profile>

<profile><id>task12</id><properties><task.dir>task12</task.dir></properties>
</profile>

<profile><id>task13</id><properties><task.dir>task13</task.dir></properties>
</profile>

<profile><id>task14</id><properties><task.dir>task14</task.dir></properties>
</profile>

<profile><id>task15</id><properties><task.dir>task15</task.dir></properties>
</profile>
  </profiles>
</project>
```